

TECHNISCHE DOKUMENTATION

Physikprojekt – Interaktiver Kräfte-Simulator

Thema: Reibungskraft und Luftwiderstand

Autor: Cristi • Schriftliches Referat + interaktiver 2D-Physiksimulator

Das Projekt online (live)

Referat: <https://notcristi.github.io/ProiectFizica/referat.html>

Simulator: <https://notcristi.github.io/ProiectFizica/index.html>

1. Worum geht es in diesem Projekt

Das Projekt hat **zwei Teile, die zusammenarbeiten**. Der erste Teil ist das **Referat** – die schriftliche Physikarbeit über Reibungskraft und Luftwiderstand, die Theorie, Formeln, Experimente und Schlussfolgerungen enthält. Der zweite Teil ist der **Simulator** – ein kleines Physiklabor, in dem man Objekte (Würfel, Kugeln, Rampen) platzieren, die Simulation starten und alle Kräfte, Geschwindigkeiten und Beschleunigungen in Echtzeit beobachten kann.

Das Besondere ist, dass beide Teile **verbunden** sind: Das Referat enthält den Simulator direkt auf der Seite, sodass man beim Lesen über die schiefe Ebene genau dort das Experiment automatisch ablaufen sehen kann. Theorie und Experiment stehen so auf derselben Seite nebeneinander.

Abbildung 1 – Der Simulator: Ein Körper auf der schiefen Ebene mit den Kräften G , N und F_f , die in Echtzeit gemessen und angezeigt werden.

2. Welche Technologien wurden verwendet

Die Anwendung läuft im Browser und wurde mit modernen Webprogrammier-Werkzeugen gebaut. Unten sind die wichtigsten Bausteine des Projekts mit ihrer jeweiligen Funktion:

Technologie	Version	Verwendungszweck
TypeScript	5.4	Die Programmiersprache des Codes (JavaScript mit Typen, strikter Modus).
Vite	5.2	Das Werkzeug, das den Entwicklungsserver startet und das Projekt bündelt.
vite-plugin-singlefile	2.0	Packt alles (JS + CSS) in eine einzige HTML-Datei am Ende.
Planck.js	1.4.3	Die Physik-Engine – berechnet Kollisionen (JS-Port von Box2D).

SheetJS (xlsx)	0.18.5	Exportiert Simulationsdaten in Excel-Dateien (.xlsx).
Canvas 2D	–	Zeichnet die Szene auf dem Bildschirm (Standard-Browser-API).
Google Fonts	–	Die Schriftarten der Benutzeroberfläche (Nunito, JetBrains Mono).

Warum diese Entscheidungen

Planck.js erledigt den schwierigen Teil – es berechnet die Kollisionen zwischen Objekten. Darüber fügt das Projekt die richtige Schulphysik hinzu (Kräfte, Reibung, Diagramme).

Vite bündelt am Ende alles in einer einzigen **HTML-Datei** (~690 KB), die sich in jedem Browser öffnet, ohne Server und ohne Installation.

TypeScript erkennt Fehler schon bevor das Programm ausgeführt wird, was den Code sicherer macht.

Es wurde kein schweres Framework (React, Vue) verwendet. Die Benutzeroberfläche wurde direkt erstellt, was das Projekt leicht und schnell hält.

3. Wie die Physik im Simulator funktioniert

Dies ist das Herz des Projekts. Die Engine berechnet die Kollisionen, und darüber wird die **echte Schulphysik** hinzugefügt.

3.1. Die sechs simulierten Kräfte

Der Simulator berechnet und zeigt sechs Kräfte an – genau die, die im Physikunterricht gelernt werden:

- **Gewichtskraft** $G = m \cdot g$
- **Normalkraft** N (Druck der Oberfläche)
- **Reibungskraft** $F_f = \mu \cdot N$
- **Luftwiderstand** $F_d = \frac{1}{2} \cdot \rho \cdot C_d \cdot A \cdot v^2$ – direkt mit dem Thema des Referats verbunden
- **Angewandte Kraft** F_{apl} , unter einem Winkel α
- **Spannung / elastische Kraft** aus Seilen, Federn und Flaschenzügen

Abbildung 1 – Die Kräfte, die auf einen Körper auf der schiefen Ebene wirken.

3.2. Zwei Koordinatensysteme

Der Browser und die Physik-Engine denken unterschiedlich: Im Browser wächst die Y-Achse nach unten, in der Physik wächst die Y-Achse nach oben. Deshalb muss jede Position bei der Umrechnung umgedreht werden (Y-Flip), sonst würden die Objekte nach oben fallen. Die Grundregel ist einfach: **1 Meter = 80 Pixel**.

Abbildung 2 – Der Unterschied zwischen Browser-Koordinaten und Physik-Koordinaten.

3.3. Warum die Simulation stabil ist (Teilschritte)

Anstatt die Physik einmal pro Bild voranzutreiben, teilt die Engine jeden Frame in **32 kleine Teilschritte** auf. Bei großen Schritten könnten schnelle Objekte durch Wände gehen oder die

Szene könnte numerisch explodieren; bei kleinen Schritten ist alles viel genauer. Es gibt außerdem eine Geschwindigkeitsbegrenzung auf 200 m/s als Sicherheitsnetz.

3.4. Echte Reibung und Normalkraft

Ein wichtiges Detail: Die auf dem Bildschirm angezeigten Werte für Normalkraft (N) und Reibung (F_f) **sind keine Schätzungen**, sondern werden **tatsächlich aus der Simulation gemessen**. Die Engine addiert bei jedem Kontakt die echten Impulse und teilt sie durch die Zeit, um die mittlere Kraft in Newton zu erhalten. So ergibt ein ruhender Körper genau $N = m \cdot g$, wie es sein soll.

3.5. Seile, Federn und Flaschenzüge

- **Das Seil** zieht, drückt aber nicht (begrenzt die maximale Distanz).
- **Die Feder** verwendet das Hookesche Gesetz $F = k \cdot \Delta l$, mit einer kleinen Dämpfung, wie eine echte Feder.
- **Der Flaschenzug** führt das Seil über einen festen Punkt (wie bei der Atwood-Maschine).

Für den Realismus stoppen Pendel aufgrund einer kleinen Reibung am Drehpunkt (nicht nur wegen der Luft), genau wie im echten Leben.

3.6. Das Projekt überprüft sich selbst

Um sicherzustellen, dass die Physik korrekt ist, führt das Projekt automatisch den Atwood-Test durch: Es vergleicht die gemessene Beschleunigung mit der theoretischen Formel $a = (m_1 - m_2) \cdot g / (m_1 + m_2)$. Wenn der Unterschied sehr klein ist, besteht der Test. Das zeigt, dass das Projekt mit Sorgfalt auf **Korrektheit** gebaut wurde und nicht nur gut aussehen soll.

4. Zeichnen, Diagramme und Datenexport

Alles wird manuell auf dem Canvas gezeichnet, bei jedem Frame: der Himmel, der Boden mit Gras, das Raster mit Metern, die Objekte und die **farbigen Kraftvektoren** (Gewichtskraft in Rot, Normalkraft in Grün, Reibung in Orange usw.), jeder mit dem Wert in Newton daneben.

Der Simulator kann ungefähr **50 verschiedene Diagramme** zeichnen (Geschwindigkeit, Beschleunigung, Energie, Kräfte, plus spezielle Kategorien für Reibung und Luftwiderstand), die sich in Echtzeit aktualisieren. Die Daten können dann **exportiert** werden:

- **Excel (.xlsx)** – alle Serien in derselben Zeitspalte, bereit zum Erstellen von Diagrammen in Excel.
- **PNG** – alle Diagramme in einem einzigen Bild, bereit zum Einfügen in Word oder PowerPoint.
- **JSON** – zum Speichern und Neuladen von Experimenten.

Abbildung 2 – Vom Simulator erstelltes Diagramm: Gesamtgeschwindigkeit als Funktion der Zeit (gerade Linie = gleichmäßig beschleunigte Bewegung).

5. Die Benutzeroberfläche und was der Nutzer tun kann

- **Zwei Modi:** Einfach (nur Masse und Schwerkraft) und Erweitert (alle Steuerungen und Diagramme).
- **Objektkatalog:** ~30 Voreinstellungen (Würfel, Kugel, Eis, Gummi, Stahlkugel, Rampen), jede mit realistischer Masse und Reibung.
- **Fertige Experimente:** Reibung, freier Fall, schiefe Ebene, Atwood, Feder, Fallschirm.

- **13 Schritt-für-Schritt gelöste Aufgaben:** Man gibt die Daten ein (Masse, Winkel, μ) und erhält die Lösung mit Formeln und automatisch erstellter Szene.
- **5 visuelle Themen** und Steuerung mit Maus, Touch (Pinch-Zoom) und Tastenkombinationen (Leertaste = Play/Pause, R = Zurücksetzen).

Abbildung 3 – Das Steuerfeld (Erweiterter Modus): Masse, Reibung μ , angewandte Kraft, Luftwiderstand und Teilschritte.

6. Wie das Projekt gebaut und gestartet wird

Der in TypeScript geschriebene Code wird von Vite überprüft und in eine einzige HTML-Datei gepackt, die dann direkt im Browser geöffnet wird, ohne Server:

Abbildung 3 – Der Weg vom Quellcode bis zur fertigen Datei.

npm install	Installiert die Abhängigkeiten (nur einmal).
npm run dev	Startet den Entwicklungsmodus mit automatischem Neuladen.
npm run build	Erstellt die endgültige Version (dist/index.html).

7. Fazit

Kurz gesagt ist das Projekt ein **Physikreferat über Reibung und Luftwiderstand**, zusammen mit einem **interaktiven 2D-Simulator**, der in TypeScript geschrieben wurde. Er verwendet die Planck.js-Engine für Kollisionen und fügt darüber die richtige Schulphysik hinzu: real gemessene Kräfte, Reibung, Luftwiderstand, Seile, Federn und Flaschenzüge. Alles wird auf dem Canvas gezeichnet, hat ungefähr 50 Diagramme und Export nach Excel, und wird in eine einzige Datei gepackt, die in jedem Browser läuft. So verbindet es die **schriftliche Theorie** mit dem **interaktiven Experiment** an einem einzigen Ort.